BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**Technical Documentation**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.03.2026	S. Köhne	Stable	Initial Release: ProTU Test Coverage Specification

1. Einleitung & Audit-Ziele

Die Process Trust Unit (ProTU) ist das automatisierte Validierungs-Framework der ProSuite Industrial. Dieses Dokument spezifiziert die genauen Test-Vektoren, mathematischen Toleranzen und physikalischen Integritätsprüfungen, die ProTU während eines Audit-Laufs (`protu_industrial ALL`) auf dem Zielsystem durchführt.

Ziel dieses Dokuments ist es, IT-Sicherheitsbeauftragten und externen Auditoren (z. B. für ISO 27001 oder TISAX) transparente Einsicht in die Metriken der Zertifizierung zu geben.

2. Modul-Spezifikationen (User-Space)

Die folgenden kryptografischen und logischen Tests werden für die Kern-Module der ProSuite durchgeführt.

2.1 ProKey (Hardware Entropy Engine)

ProKey ist für die Erzeugung von physikalischem Zufall (Entropie) zuständig. Ein Versagen hier kompromittiert alle nachgelagerten Schlüssel.

- **Sanity & Starvation Check:** Prüft, ob der Zufallsgenerator "tot" ist (Rückgabe von ausschließlich Null-Bytes).
- **Repetition Check (Stuck-At Faults):** Generiert sequenzielle Schlüssel (K_1, K_2, K_3) und stellt sicher, dass kein Schlüssel dem vorherigen gleicht ($K_n \neq K_{n+1}$). Dies deckt Hardware-Loops auf.
- **Statistical Monobit Test:** Generiert ein 4-Kilobyte-Sample (32.768 Bits) und misst die exakte Verteilung von Einsen und Nullen. Der Test erzwingt eine strikte industrietaugliche Gleichverteilung (Toleranzbereich: 48 % bis 52 %). Fällt der Wert aus diesem Rahmen, wird auf einen "Biased Output" (manipulierter Zufall) geschlossen und das Audit schlägt fehl.

2.2 ProHash (Deterministic Integrity Engine)

ProHash liefert den unbestechlichen kryptografischen Fingerabdruck für Dateien und Datenströme.

- **Determinismus-Prüfung:** Verifiziert, dass exakt identische Eingaben mathematisch zwingend denselben 256-Bit-Hash erzeugen, ohne von Speicherzuständen beeinflusst zu werden.
- **Avalanche-Effekt (Diffusion):** Ein einzelnes Bit in einem 64-Byte-Datenblock wird gekippt. ProTU verifiziert, dass sich daraufhin statistisch ca. 50 % (Toleranz: 35 % bis 65 %) der resultierenden 256 Hash-Bits ändern. Dies beweist die Nicht-Umkehrbarkeit.
- **Streaming Consistency (Sponge-Test):** Beweist die interne Puffer-Sicherheit. Der Test verifiziert, dass das Hashen eines kompletten Strings dasselbe Ergebnis liefert wie das Hashen desselben Strings, der in asynchronen Chunks (Fragmenten) übergeben wird.
- **Kollisions-Resistenz:** Basisprüfung gegen triviale Hash-Kollisionen bei minimal abweichenden Strings.

2.3 ProED (Encryption Core)

Der Kernalgorithmus für die symmetrische Hochgeschwindigkeits-Verschlüsselung.

- **Basic Roundtrip:** Verschlüsselt einen Datensatz im Speicher und entschlüsselt ihn wieder. Das Resultat muss bitgenau dem Original entsprechen; der Chiffrat-Zwischenstand muss zwingend abweichen.
- **IV Impact Analysis:** Zwei identische Datensätze werden mit demselben Master-Key, aber einem nur um 1 Byte abweichenden Initialisierungsvektor (IV) verschlüsselt. ProTU stellt sicher, dass daraus zwei völlig isolierte, unkorrelierte Cipher-Streams entstehen (Verhinderung von Two-Time-Pad-Angriffen).
- **Context Ratcheting & Isolation:** Prüft den Schutz gegen ECB-Muster (Electronic Codebook). Zwei identische Klartextblöcke, die nacheinander in denselben Stream gespeist werden, müssen unterschiedliche Chiffrate erzeugen.

2.4 ProChat (Zero-Trust Protocol)

Die Anwendungsschicht für dezentrale P2P-Kommunikation.

- **Tamper Protection (AEAD / HMAC):** Ein verschlüsseltes Paket wird auf Bitebene gezielt manipuliert ("Bitflip-Attacke" zur Simulation eines Man-in-the-Middle). ProTU verifiziert, dass der Empfänger dieses Paket strikt abweist (Auth-Error), anstatt korrumpierte Daten zu verarbeiten.
- **Replay-Attack Protection:** Ein valides, erfolgreich entschlüsseltes Paket wird vom Test-Framework abgefangen und ein zweites Mal an den Empfänger gesendet. Der Test gilt nur als bestanden, wenn das System die Duplikation erkennt und das Paket blockiert.

3. Black-Box & System Integration Tests

Neben den C-Bibliotheken prüft ProTU auch die kompilierten Kommandozeilen-Tools (CLI) und Web-Bridges (WASM) wie sie der Endanwender nutzt.

- **CLI Parameter Handling:** Die Tools (z.B. `proedc.exe`) werden gezielt mit fehlenden Schlüsseln, nicht existierenden Dateien und falschen Längenangaben aufgerufen. ProTU verifiziert, dass die Tools sichere Exit-Codes (z. B. `1` oder `2`) zurückgeben und keine kritischen Systemfehler auslösen.
- **WASM Boundary Defense:** Im WebAssembly-Heap werden Null-Pointer-Injektionen und Pufferüberläufe simuliert, um sicherzustellen, dass manipulierte JavaScript-Aufrufe aus dem Browser die kryptografische Engine nicht zum Absturz (Segfault) bringen können.

4. ProTU Meta-Validation (Self-Test)

Ein Audit-Werkzeug muss seine eigene Integrität beweisen können. Zu Beginn jedes Audits führt ProTU einen Selbsttest durch:

- **Counter Mechanics:** Verifiziert, dass die internen Metriken (Passed/Failed Zähler) mathematisch konsistent arbeiten.
- **Logging Overflow Protection:** Ein massiver, übergroßer String wird in die Logging-Engine injiziert, um zu beweisen, dass der Report-Generator immun gegen Pufferüberläufe ist.
- **Dual-Seal Linkage:** Prüft die kryptografische Verbindung zur ProHash-Engine, welche für die finale Versiegelung (Manipulationsschutz) des `.brep` Audit-Reports benötigt wird.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**

System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**